

Optical Modeling of Thermochemical VO₂-Based Oxide Nanocomposites

Alexandre Autran¹

Supervised by Prof. Ngoc Duy Nguyen² and Dr. Amaury Baret²

¹Department of Mathematics, École Normale Supérieure, Rennes, France

²Department of Physics, University of Liège, Liège, Belgium

Abstract— This report develops three linked models for thermochemical VO₂-based smart-window coatings: a transfer-matrix calculation of multilayer spectra and integrated metrics; a stand-alone Python implementation of the Li–Niklasson–Granqvist nanothermochemical model for spheroidal VO₂ inclusions using Maxwell–Garnett theory; and its integration into `chromasolver.py`, where compact and nanocomposite layers share one optimization workflow. The code reproduces the reference spectral and integrated trends and the optimal-thickness region of a published TiO₂/VO₂/TiO₂ trilayer. Trends and optimal geometries agree well; systematic offsets in absolute transmittance remain and are interpreted in light of the available optical data.

Acknowledgments— I thank Prof. Ngoc Duy Nguyen and Dr. Amaury Baret for supervising this internship and for their guidance and support.

Keywords: thermochemical, VO₂, smart windows, transfer-matrix method, nanocomposites, Maxwell–Garnett theory, optical optimization, NSGA-II.

Table of Contents

1	Thermochromic smart windows and optical metrics	3			
1.1	Main notation	3		A.3.2	Objectives and constraints 18
1.2	Thermochromic materials for smart windows	3		A.3.3	Optical metadata and state semantics . 19
1.3	Spectral balance and emissivity	4		A.3.4	NSGA-II operators and termination . . 19
1.4	Weighted optical metrics	4		A.4	Candidate representation and population evaluation 20
1.4.1	Luminous transmittance	4		A.4.1	Detection and reconstruction of variable thicknesses 20
1.4.2	Solar transmittance and solar modulation	5		A.4.2	Independent evaluation and multiprocessing 20
1.4.3	Transparency–modulation figure of merit	5		A.5	Stack construction and coherent–incoherent dispatch 20
1.4.4	Long-wave infrared emissivity	6		A.6	Optical input data and preprocessing 21
2	Multilayer model and ChromaSolver	6		A.6.1	Input files 21
2.1	Transfer-matrix model	6		A.6.2	Interpolation and precomputed arrays . 22
2.2	Numerical and optimization workflow	8		A.7	Spectral calculation and returned quantities . . 22
3	Contributions and validation	9		A.7.1	Angles, polarizations, and target layer . 22
3.1	Work completed during the internship	9		A.7.2	Transfer-matrix dispatch and layer-resolved absorption 23
3.2	Benchmark: an A-TiO ₂ /VO ₂ /R-TiO ₂ trilayer	9		A.8	Serialization and post-processing 23
3.3	Independent nanothermochromic model	10		A.8.1	Saved spectral data 23
3.3.1	Optical input and particle dielectric function	10		A.8.2	Polarization combination 24
3.3.2	Maxwell–Garnett effective medium	11		A.8.3	LWIR integration and final metric dictionary 24
3.3.3	Spheroidal depolarization factors	11		A.9	Conversion to optimizer arrays and failure handling 25
3.3.4	Coherent transmittance of one effective layer	12		A.10	Persistent history, analysis tools, and run-control flags 25
3.4	Validation of implementation	12			
3.5	Generalization of ChromaSolver	14		B	Genetic algorithms and NSGA-II 26
3.5.1	Main corrections and implementation choices	14		B.1	Optimization problem and population 26
3.5.2	Validation with integrated metrics	15		B.2	Canonical generation 27
4	Conclusion	17		B.2.1	Simulated binary crossover 27
A	Detailed software architecture and execution workflow	17		B.2.2	Polynomial mutation 27
A.1	Purpose and scope	17		B.3	Multiobjective dominance 28
A.2	Modules and data products	17		B.4	NSGA-II selection 28
A.3	Representative optimization configuration	18		B.5	Constraints, budget, and stopping 29
A.3.1	Thickness template and bounds	18		References	30

1 Thermochromic smart windows and optical metrics

1.1 Main notation

Table 1: Main notation used in this report.

Symbol	Meaning
λ	Vacuum wavelength.
$\Phi_s, \Phi_r, \Phi_a, \Phi_t$	Incident, reflected, absorbed, and transmitted spectral fluxes.
$T(\lambda), R(\lambda), A(\lambda)$	Spectral transmittance, reflectance, and absorptance.
$\varepsilon(\lambda)$	Spectral emissivity. At thermal equilibrium, for the same direction and polarization, $\varepsilon = A$.
$\phi(\lambda)$	Photopic sensitivity used for visible weighting.
$\psi(\lambda)$	AM1.5 solar spectral irradiance used for solar weighting.
T_{lum}^τ	Luminous transmittance in thermal state $\tau \in \{\text{cold}, \text{hot}\}$.
T_{sol}^τ	Solar transmittance in thermal state τ .
ΔT_{sol}	Solar modulation, $T_{\text{sol}}^{\text{cold}} - T_{\text{sol}}^{\text{hot}}$.
FOM	Combined metric, $\Delta T_{\text{sol}} (T_{\text{lum}}^{\text{min}})^4$.
$M_{\text{BB}}(\lambda, T)$	Blackbody spectral exitance at temperature T .
$\bar{\varepsilon}_{\text{LWIR}}$	Blackbody-weighted emissivity on $[8, 13] \mu\text{m}$, evaluated in either thermal state.
$\Delta \bar{\varepsilon}_{\text{LWIR}}$	LWIR emissivity modulation, $\bar{\varepsilon}_{\text{LWIR}}^{\text{hot}} - \bar{\varepsilon}_{\text{LWIR}}^{\text{cold}}$.

1.2 Thermochromic materials for smart windows

Thermochromic materials reversibly change their optical response with temperature without electrical control, enabling passive regulation of transmitted, reflected, and absorbed solar radiation [1, 2, 3, 4]. Bulk undoped VO_2 undergoes an insulator-to-metal transition near $T_c \simeq 68^\circ\text{C}$: below T_c it is insulating and monoclinic, and above it metallic and rutile. The transition strongly modifies the near-infrared optical constants; doping can shift the transition temperature [2, 3, 5].

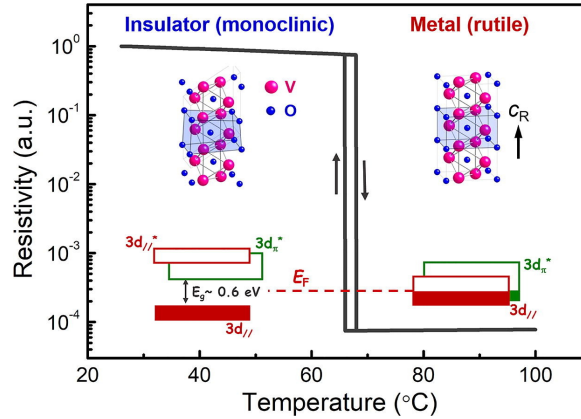


Figure 1: Reversible phase transition of VO_2 . Adapted from Liu et al. [5].

For a smart-window coating, the main design goals are

$$T_{\text{lum}}^{\text{min}} \text{ large, } \Delta T_{\text{sol}} \text{ large and positive, } \bar{\varepsilon}_{\text{LWIR}}^\tau \text{ small for both thermal states.}$$

These objectives conflict: a thicker active layer can increase solar modulation while reducing visible transmittance, and a metallic layer can lower thermal emissivity while absorbing visible light. The complete stack must therefore be optimized jointly [6, 7].

1.3 Spectral balance and emissivity

At wavelength λ , energy conservation gives

$$\Phi_s(\lambda) = \Phi_r(\lambda) + \Phi_a(\lambda) + \Phi_t(\lambda).$$

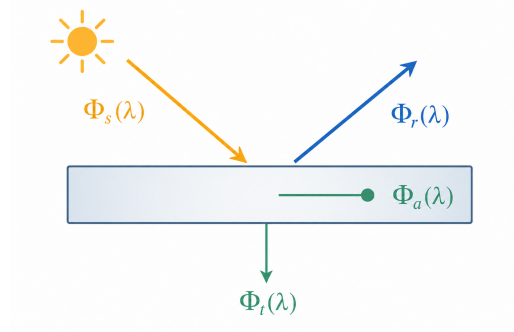


Figure 2: Incident, reflected, absorbed, and transmitted spectral fluxes.

Define

$$T(\lambda) := \frac{\Phi_t(\lambda)}{\Phi_s(\lambda)}, \quad R(\lambda) := \frac{\Phi_r(\lambda)}{\Phi_s(\lambda)} \quad \text{and} \quad A(\lambda) := \frac{\Phi_a(\lambda)}{\Phi_s(\lambda)}.$$

For a passive stack, $0 \leq T, R, A \leq 1$ and $T(\lambda) + R(\lambda) + A(\lambda) = 1$. At thermal equilibrium, Kirchhoff's law gives the directional and polarization-dependent relation $\varepsilon(\lambda, \theta, p) = A(\lambda, \theta, p)$. At normal incidence, we omit θ and p , so

$$\varepsilon(\lambda) = 1 - R(\lambda) - T(\lambda). \quad (1)$$

If the stack is opaque in the thermal infrared, $T(\lambda) \simeq 0$, and $\varepsilon(\lambda) \simeq 1 - R(\lambda)$.

1.4 Weighted optical metrics

For an integrable spectrum g , a non-negative weight w , and an interval $[a, b]$, define

$$\mathcal{W}_{[a,b]}[g; w] := \frac{\int_a^b g(\lambda) w(\lambda) d\lambda}{\int_a^b w(\lambda) d\lambda}.$$

1.4.1 Luminous transmittance

The code defines

$$T_{\text{lum}}^\tau := \mathcal{W}_{[380 \text{ nm}, 780 \text{ nm}]}[T^\tau; \phi], \quad \text{with } \tau \in \{\text{cold}, \text{hot}\}.$$

With $\lambda_{\text{nm}} := \lambda / (1 \text{ nm})$, the photopic sensitivity used in the calculations is approximated by

$$\phi(\lambda) \simeq 1.014 \exp \left[-\frac{1}{2} \left(\frac{\ln(\lambda_{\text{nm}}) - \ln(556.3)}{0.075} \right)^2 \right]. \quad (2)$$

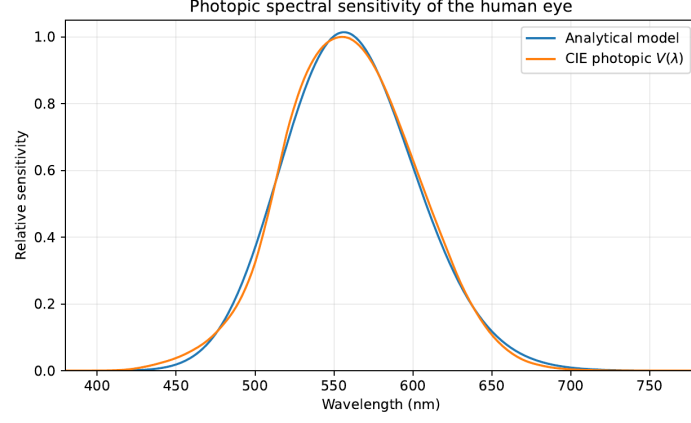


Figure 3: Photopic sensitivity and the analytical approximation used in [equation \(2\)](#).

All equations use fractions in $[0, 1]$; percentages are only display values: $T_{\text{lum}}^{\tau}(\%) := 100T_{\text{lum}}^{\tau}$. We impose $T_{\text{lum}}^{\min} \geq 0.70$; the cited high-performance region is $T_{\text{lum}} \simeq 0.75$ and $\Delta T_{\text{sol}} \simeq 0.20$ [\[2, 3, 6\]](#).

1.4.2 Solar transmittance and solar modulation

The solar transmittance is $T_{\text{sol}}^{\tau} := \mathcal{W}_{[380 \text{ nm}, 2500 \text{ nm}]}[T^{\tau}; \psi]$, where ψ is the AM1.5 spectrum used numerically. Solar modulation is $\Delta T_{\text{sol}} := T_{\text{sol}}^{\text{cold}} - T_{\text{sol}}^{\text{hot}}$. The usual thermochromic response has $\Delta T_{\text{sol}} > 0$.

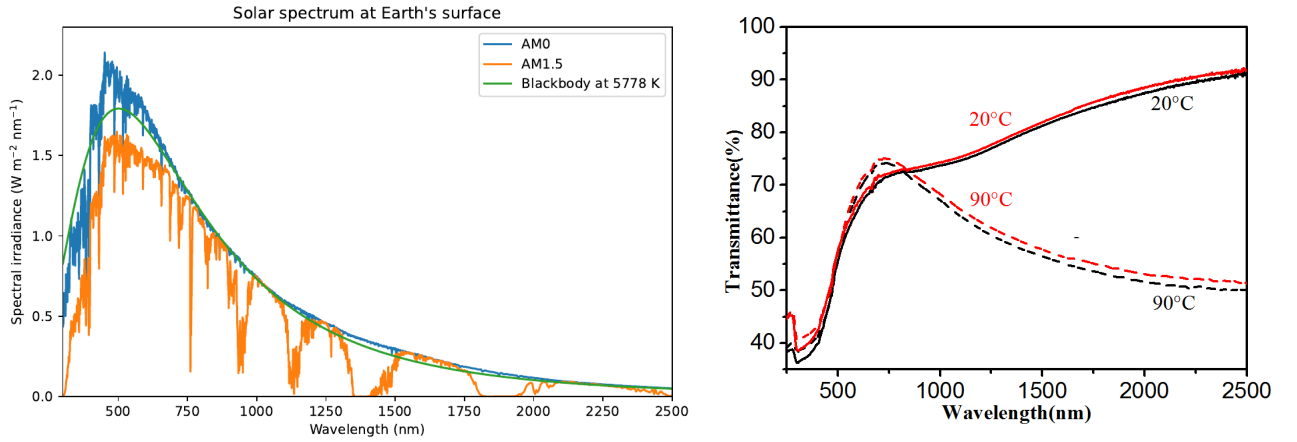


Figure 4: AM1.5 solar spectrum used for weighting (left) and typical cold- and hot-state transmittance spectra of VO_2 (right).

1.4.3 Transparency–modulation figure of merit

We define

$$T_{\text{lum}}^{\min} := \min \{ T_{\text{lum}}^{\text{cold}}, T_{\text{lum}}^{\text{hot}} \} \quad \text{and} \quad \text{FOM} := \Delta T_{\text{sol}} (T_{\text{lum}}^{\min})^4. \quad (3)$$

The fourth power strongly penalizes low visible transmittance. For percentage-valued Python outputs, [equation \(3\)](#) becomes

$$\text{FOM} = \frac{\Delta T_{\text{sol}}(\%)}{100} \left[\frac{\min \{ T_{\text{lum}}^{\text{cold}}(\%), T_{\text{lum}}^{\text{hot}}(\%) \}}{100} \right]^4.$$

1.4.4 Long-wave infrared emissivity

The blackbody spectral exitance is

$$M_{\text{BB}}(\lambda, T) := \frac{2\pi hc^2}{\lambda^5} \frac{1}{\exp\left(\frac{hc}{\lambda k_B T}\right) - 1}.$$

For any interval $[\lambda_a, \lambda_b]$, define

$$\bar{\epsilon}_{[\lambda_a, \lambda_b]}(T) := \mathcal{W}_{[\lambda_a, \lambda_b]}[\epsilon(\cdot); M_{\text{BB}}(\cdot, T)].$$

The LWIR emissivity used in the code is

$$\bar{\epsilon}_{\text{LWIR}} := \bar{\epsilon}_{[8\text{ }\mu\text{m}, 13\text{ }\mu\text{m}]}(300\text{ K}).$$

It covers the atmospheric long-wave window and appears as `e_LWIR` in post-processing. For two thermal states, $\Delta\bar{\epsilon}_{\text{LWIR}} := \bar{\epsilon}_{\text{LWIR}}^{\text{hot}} - \bar{\epsilon}_{\text{LWIR}}^{\text{cold}}$. It uses the normal-incidence directional emissivity in [equation \(1\)](#).

2 Multilayer model and ChromaSolver

2.1 Transfer-matrix model

Represent the stack by $\mathcal{S} := 0/1/\dots/N/s$, where 0 and s are the incident and exit media and $j = 1, \dots, N$ indexes finite layers. Layer j has thickness d_j and complex refractive index

$$\tilde{n}_j(\lambda) := n_j(\lambda) + ik_j(\lambda), \quad k_j(\lambda) \geq 0.$$

The geometry is fixed across thermal states; only the VO_2 optical constants change:

$$\tilde{n}_{\text{VO}_2}^{\text{cold}}(\lambda) \neq \tilde{n}_{\text{VO}_2}^{\text{hot}}(\lambda).$$

For a plane wave of vacuum wavelength λ , Snell's law gives

$$\tilde{n}_j(\lambda) \sin \theta_j = \tilde{n}_0(\lambda) \sin \theta_0.$$

At normal incidence, $\theta_j = 0$ for every layer. For a coherent layer,

$$\delta_j(\lambda) := \frac{2\pi}{\lambda} \tilde{n}_j(\lambda) d_j \cos \theta_j.$$

With the admittance convention

$$q_j^{(s)} := \tilde{n}_j \cos \theta_j \quad \text{and} \quad q_j^{(p)} := \frac{\tilde{n}_j}{\cos \theta_j},$$

the characteristic matrix is

$$M_j := \begin{pmatrix} \cos \delta_j & \frac{i}{q_j} \sin \delta_j \\ iq_j \sin \delta_j & \cos \delta_j \end{pmatrix}.$$

The ordered product is

$$M := M_1 M_2 \dots M_N = \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{pmatrix}.$$

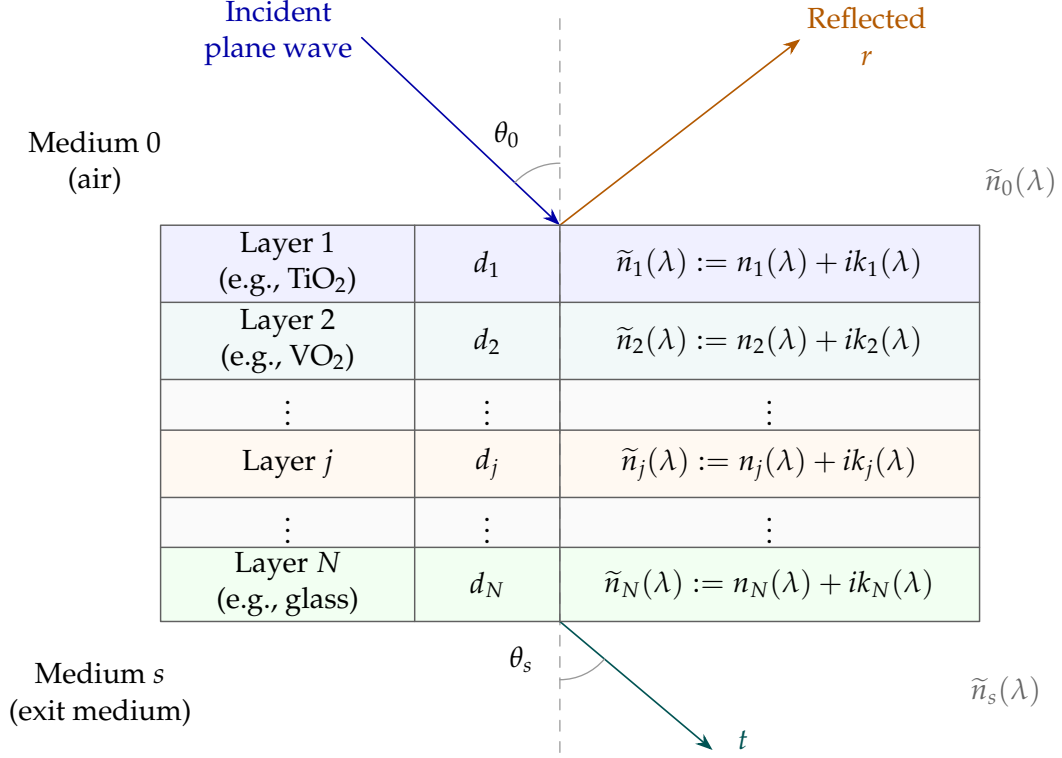
The amplitude coefficients are

$$r := \frac{q_0 M_{11} + q_0 q_s M_{12} - M_{21} - q_s M_{22}}{q_0 M_{11} + q_0 q_s M_{12} + M_{21} + q_s M_{22}} \quad \text{and} \quad t := \frac{2q_0}{q_0 M_{11} + q_0 q_s M_{12} + M_{21} + q_s M_{22}}.$$

For non-absorbing incident and exit media,

$$R := |r|^2, \quad T := \frac{\operatorname{Re}(q_s)}{\operatorname{Re}(q_0)} |t|^2 \quad \text{and} \quad A := 1 - R - T.$$

If the incident or exit medium is absorbing, the power flux must be computed with the more general conventions implemented by the solver [8, 9].



Example stack (top to bottom): Air / TiO₂ / VO₂ / Ag / glass / Air

Figure 5: Multilayer optical stack model. Some layers may be coherent (thin) and others incoherent (thick or rough). Incoherent layers are handled with the incoherent transfer-matrix formalism.

The Python package `tmm` uses

```
coh_tmm(pol, n_list, d_list, th_0, lam_vac)
```

for fully coherent stacks and

```
inc_tmm(pol, n_list, d_list, c_list, th_0, lam_vac)
```

for mixed coherent–incoherent stacks. Thin oxide and metallic films are normally coherent; treating thick glass as incoherent suppresses artificial fringes. The functions `absorp_in_each_layer` and `inc_absorp_in_each_layer` return layer-resolved absorption.

For each thermal state $\tau \in \{\text{cold}, \text{hot}\}$, the complete optical map is

$$\left\{ \tilde{n}_j^\tau(\lambda), d_j \right\}_{j=1}^N \longmapsto \{T^\tau(\lambda), R^\tau(\lambda), A^\tau(\lambda)\}.$$

The spectra are then reduced to the metrics of [section 1](#).

2.2 Numerical and optimization workflow

The SPIN optimization framework calls the optical model through `chromasolver.py`. A candidate geometry is

$$\mathbf{x} := (x_1, \dots, x_{n_{\text{var}}}) \in \mathcal{X} := \prod_{i=1}^{n_{\text{var}}} [x_i^{\min}, x_i^{\max}].$$

It is inserted into a complete thickness vector $\mathbf{d}_{\text{full}} := (d_1, \dots, d_N)$. The multiobjective problem has the form

$$\min_{\mathbf{x} \in \mathcal{X}} F(\mathbf{x}) := (f_1(\mathbf{x}), \dots, f_{n_{\text{obj}}}(\mathbf{x})) \quad \text{subject to} \quad G_j(\mathbf{x}) \leq 0, \quad j = 1, \dots, n_c.$$

Here n_{var} , n_{obj} , and n_c count variables, objectives, and constraints. NSGA-II is called through `pymoo`; maximization is converted by sign reversal:

$$\max_{\mathbf{x}} T_{\text{lum}}(\mathbf{x}) \iff \min_{\mathbf{x}} [-T_{\text{lum}}(\mathbf{x})].$$

A lower bound $T_{\text{lum}} \geq T_{\text{target}}$ is written as $G(\mathbf{x}) := T_{\text{target}} - T_{\text{lum}}(\mathbf{x}) \leq 0$.

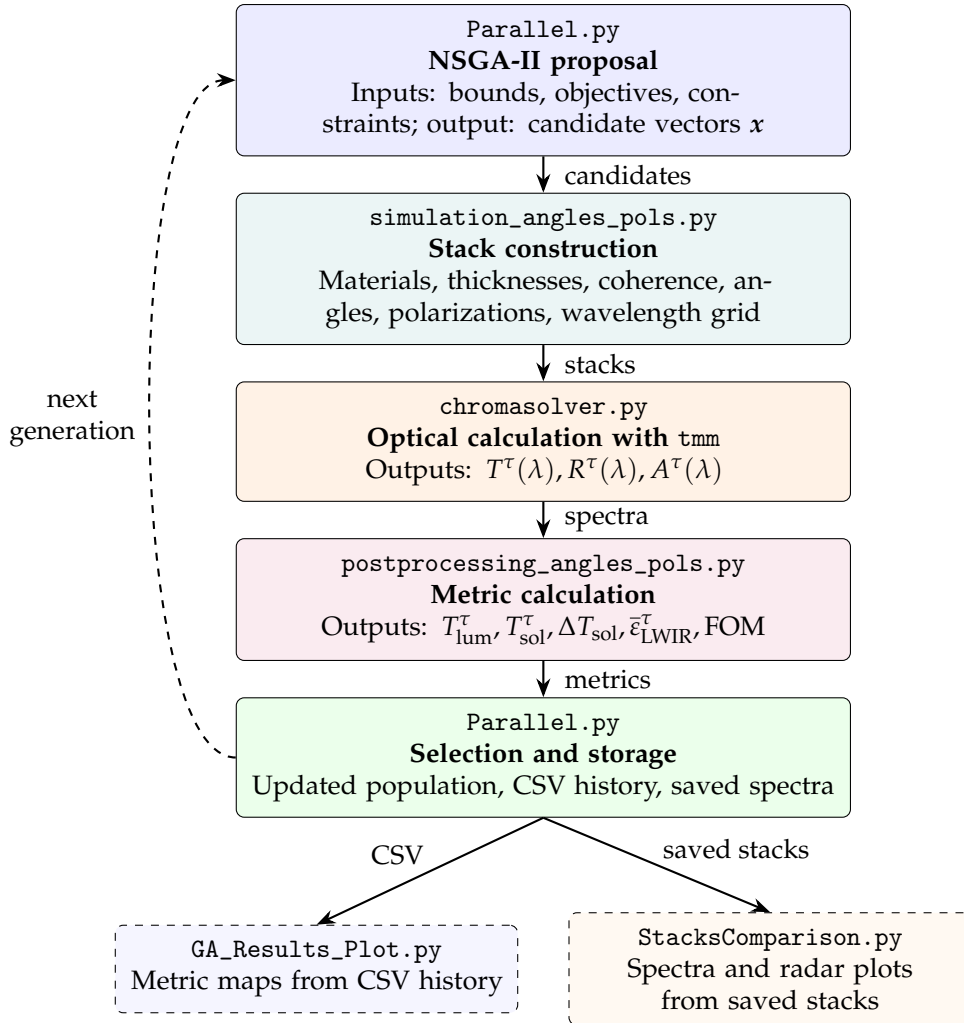


Figure 6: General workflow of the multilayer optimization code.

For one candidate, the computational chain is

$$\mathbf{x} \mapsto \mathbf{d}_{\text{full}} \mapsto \mathcal{S} \mapsto \{T^\tau, R^\tau, A^\tau\}_{\tau \in \{\text{cold}, \text{hot}\}} \mapsto \mathbf{q} \mapsto \{F, G\}.$$

A consistent metric vector is

$$\mathbf{q} := \left(T_{\text{lum}}^{\text{cold}}, T_{\text{lum}}^{\text{hot}}, T_{\text{sol}}^{\text{cold}}, T_{\text{sol}}^{\text{hot}}, \Delta T_{\text{sol}}, \bar{\epsilon}_{\text{LWIR}}^{\text{cold}}, \bar{\epsilon}_{\text{LWIR}}^{\text{hot}}, \Delta \bar{\epsilon}_{\text{LWIR}}, \text{FOM} \right).$$

In the source, E equals $A = 1 - R - T$; the report uses A to avoid confusion with electric field or energy. Once an effective index $\tilde{n}_{\text{eff}}(\lambda)$ is known, a nanocomposite layer enters the transfer-matrix solver exactly like a homogeneous layer:

$$(\tilde{n}_{\text{eff}}(\lambda), d) \mapsto \mathbf{tmm}.$$

Thus the effective-medium model enters multilayer optimization without changing the solver; software architecture, execution, data handling, and post-processing are documented in [section A](#).

3 Contributions and validation

3.1 Work completed during the internship

The internship comprised three phases:

1. Review thermochromic glazing and the existing Python workflow, then reproduce a published $\text{TiO}_2/\text{VO}_2/\text{TiO}_2$ benchmark.
2. Implement the Li–Niklasson–Granqvist nanothermochromic model in the stand-alone program `nanochromic.py` [10].
3. Generalize `chromasolver.py` to select the dielectric matrix, particle aspect ratio, and orientation directly.

3.2 Benchmark: an A– $\text{TiO}_2/\text{VO}_2/\text{R–TiO}_2$ trilayer

Becker et al. optimized anatase $\text{TiO}_2/50\text{ nm VO}_2/\text{rutile TiO}_2$ on glass [3]. Their ideal optimum is $\mathbf{d}_{\text{ref}} := (178, 50, 152)\text{ nm}$, ordered as $(d_{\text{AR}}, d_{\text{VO}_2}, d_{\text{buffer}})$. ChromaSolver gives $\mathbf{d}_{\text{CS}} := (180, 50, 150)\text{ nm}$, and $\|\mathbf{d}_{\text{CS}} - \mathbf{d}_{\text{ref}}\|_{\infty} = 2\text{ nm}$.

Table 2: Published values and ChromaSolver values for the trilayer benchmark. Approximate published values are read from figures.

Method	d_{AR} (nm)	d_{VO_2} (nm)	d_{buf} (nm)	$T_{\text{lum}}^{\text{cold}}$ ideal (%)	$T_{\text{lum}}^{\text{cold}}$ rough. (%)	$T_{\text{lum}}^{\text{hot}}$ (%)	$T_{\text{sol}}^{\text{cold}}$ (%)	$T_{\text{sol}}^{\text{hot}}$ (%)	ΔT_{sol} (%)	FOM
Becker et al.	178	50	152	~ 45	$\sim 35\text{--}37$	–	–	–	$\sim 15\text{--}16$	–
ChromaSolver	180	50	150	60.84	not modeled	59.47	53.38	39.85	13.52	0.0169

For the ChromaSolver optimum, $\text{FOM} = 0.1352 (0.5947)^4 \simeq 0.0169$.

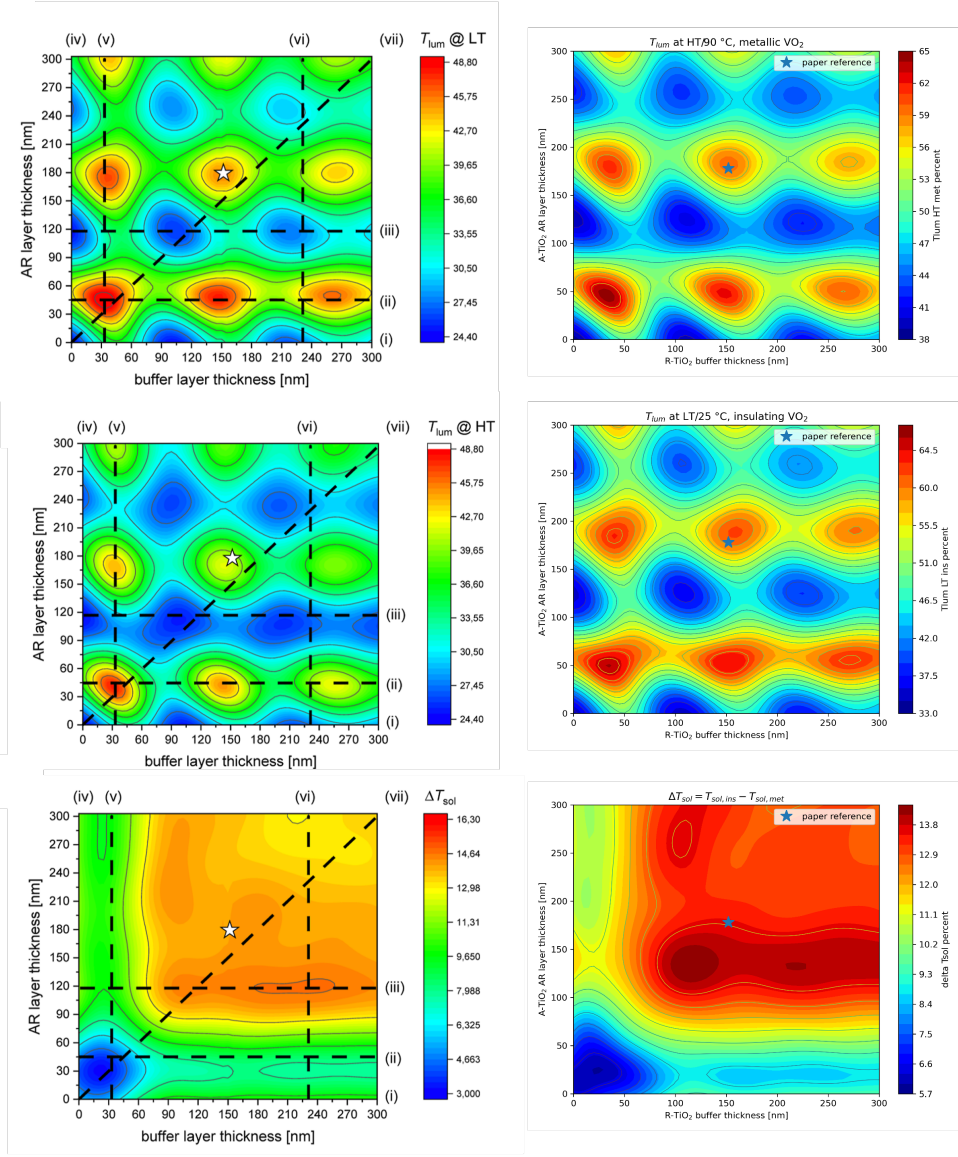


Figure 7: Contour maps for the A–TiO₂/VO₂/R–TiO₂ trilayer with $d_{\text{VO}_2} = 50$ nm. The left column is from Becker et al. [3]; the right column is obtained with ChromaSolver. The thermal states appear in a different vertical order in the two columns.

Contour geometry and the optimum region agree, supporting the interference model and search. Absolute luminous transmittance differs by about 16 percentage points from the published ideal-interface value and more from the roughness-aware value, plausibly because of optical constants, substrate treatment, spectral weights, or interface modeling.

3.3 Independent nanothermochromic model

Li, Niklasson, and Granqvist model dilute subwavelength spherical or spheroidal VO₂ inclusions in a transparent dielectric matrix [10]; depolarization factors encode shape and Maxwell–Garnett theory gives the effective dielectric function.

3.3.1 Optical input and particle dielectric function

For material phase $\sigma \in \{\text{ins}, \text{met}\}$, let

$$\tilde{n}_p^\sigma(\lambda) := n_p^\sigma(\lambda) + ik_p^\sigma(\lambda), \quad k_p^\sigma(\lambda) \geq 0.$$

The particle dielectric function is

$$\varepsilon_p^\sigma(\lambda) := \left[\tilde{n}_p^\sigma(\lambda) \right]^2 = \left[\left(n_p^\sigma(\lambda) \right)^2 - \left(k_p^\sigma(\lambda) \right)^2 \right] + 2i n_p^\sigma(\lambda) k_p^\sigma(\lambda).$$

For metallic nanoparticles, a finite-size Drude correction can be written as

$$\varepsilon_p = \varepsilon_f - \varepsilon_f^D + \varepsilon_p^D, \quad \varepsilon_v^D(\omega) := 1 - \frac{\omega_{p_v}^2}{\omega(\omega + i/\tau_v)}, \quad v \in \{f, p\}.$$

The subscripts f and p denote film and particle Drude responses. The reference assumes $\omega_{pf} = \omega_{pp}$ and $\tau_p^{-1} = \tau_f^{-1} + \frac{2v_F}{D_p}$. Because metallic VO₂ already has a short scattering time, it sets $\varepsilon_p = \varepsilon_f$; we retain this approximation [10].

3.3.2 Maxwell–Garnett effective medium

Let f be the particle volume fraction and ε_m the matrix dielectric function. For one principal direction with depolarization factor L , define

$$\alpha_L(\lambda) := \frac{\varepsilon_p(\lambda) - \varepsilon_m(\lambda)}{\varepsilon_m(\lambda) + L[\varepsilon_p(\lambda) - \varepsilon_m(\lambda)]}.$$

For an oriented ensemble probed along this direction,

$$\varepsilon_{\text{eff}}(\lambda) := \varepsilon_m(\lambda) \frac{1 + \frac{2}{3}f\alpha_L(\lambda)}{1 - \frac{1}{3}f\alpha_L(\lambda)}. \quad (4)$$

For random orientations,

$$\bar{\alpha}(\lambda) := \frac{1}{3} \sum_{j=1}^3 \frac{\varepsilon_p(\lambda) - \varepsilon_m(\lambda)}{\varepsilon_m(\lambda) + L_j[\varepsilon_p(\lambda) - \varepsilon_m(\lambda)]}, \quad (5)$$

and α_L in [equation \(4\)](#) is replaced by $\bar{\alpha}$. Thus polarizabilities, not depolarization factors, are averaged. The effective index is the passive square root

$$\tilde{n}_{\text{eff}}(\lambda) := \sqrt{\varepsilon_{\text{eff}}(\lambda)}, \quad \text{Im}[\tilde{n}_{\text{eff}}(\lambda)] \geq 0.$$

The model assumes dilute, subwavelength, weakly interacting inclusions in an isotropic host, with reference parameters

$$f = 0.01, \quad \varepsilon_m = 2.25, \quad n_{\text{sub}} = 1.5, \quad d = 5 \mu\text{m}.$$

The equivalent compact-VO₂ thickness is therefore $fd = 0.01 \times 5 \mu\text{m} = 0.05 \mu\text{m}$.

3.3.3 Spheroidal depolarization factors

Let $a_{\parallel}$ and $a_{\perp}$ be the symmetry-axis and transverse semi-axes, with $m := \frac{a_{\parallel}}{a_{\perp}}$. Then $m > 1$, $m = 1$, and $0 < m < 1$ denote prolate, spherical, and oblate particles. Their depolarization factors $L_{\parallel}$ and $L_{\perp}$ satisfy $L_{\parallel} + 2L_{\perp} = 1$. For $m > 1$, set $e := \sqrt{1 - m^{-2}}$, and

$$L_{\parallel} := \frac{1 - e^2}{2e^3} \left[\ln \left(\frac{1 + e}{1 - e} \right) - 2e \right].$$

For $0 < m < 1$, set $\xi := \sqrt{m^{-2} - 1}$, and

$$L_{\parallel} := \frac{1 + \xi^2}{\xi^3} (\xi - \arctan \xi).$$

In both cases,

$$L_{\perp} := \frac{1 - L_{\parallel}}{2}, \quad L_{\parallel} = L_{\perp} = \frac{1}{3} \quad \text{when } m = 1.$$

For normally incident light and symmetry axes normal to the substrate, the electric field is in-plane and the oriented response is transverse: $L := L_{\perp}$. Random orientations use $(L_{\parallel}, L_{\perp}, L_{\perp})$ in [equation \(5\)](#).

3.3.4 Coherent transmittance of one effective layer

Consider air, a layer of thickness d , and a non-absorbing substrate with indices N_0 , $N_1(\lambda)$, and N_{sub} . At normal incidence, $\delta(\lambda) := \frac{2\pi N_1(\lambda)d}{\lambda}$. The interface coefficients are

$$r_{01} := \frac{N_0 - N_1}{N_0 + N_1}, \quad r_{12} := \frac{N_1 - N_{\text{sub}}}{N_1 + N_{\text{sub}}}, \quad t_{01} := \frac{2N_0}{N_0 + N_1} \quad \text{and} \quad t_{12} := \frac{2N_1}{N_1 + N_{\text{sub}}}.$$

Summing all coherent internal reflections gives [\[9\]](#)

$$t(\lambda) = \frac{t_{01}t_{12}\exp(i\delta)}{1 + r_{01}r_{12}\exp(2i\delta)}$$

and $T(\lambda) = \frac{\text{Re}(N_{\text{sub}})}{\text{Re}(N_0)} |t(\lambda)|^2$. For a compact film, $N_1 = \tilde{n}_p$. For a nanocomposite, $N_1 = \tilde{n}_{\text{eff}}$. The full calculation is

$$\lambda \mapsto (n_p^{\sigma}(\lambda), k_p^{\sigma}(\lambda)) \mapsto \varepsilon_p^{\sigma}(\lambda) \mapsto (L_{\parallel}, L_{\perp}) \mapsto \varepsilon_{\text{eff}}^{\sigma}(\lambda) \mapsto \tilde{n}_{\text{eff}}^{\sigma}(\lambda) \mapsto T^{\sigma}(\lambda).$$

The reference aspect ratios are $m \in \{0.1, 0.2, 0.5, 1, 2, 5, 10\}$.

3.4 Validation of implementation

The model was first isolated in `nanochromic.py` before integration into `ChromaSolver`.

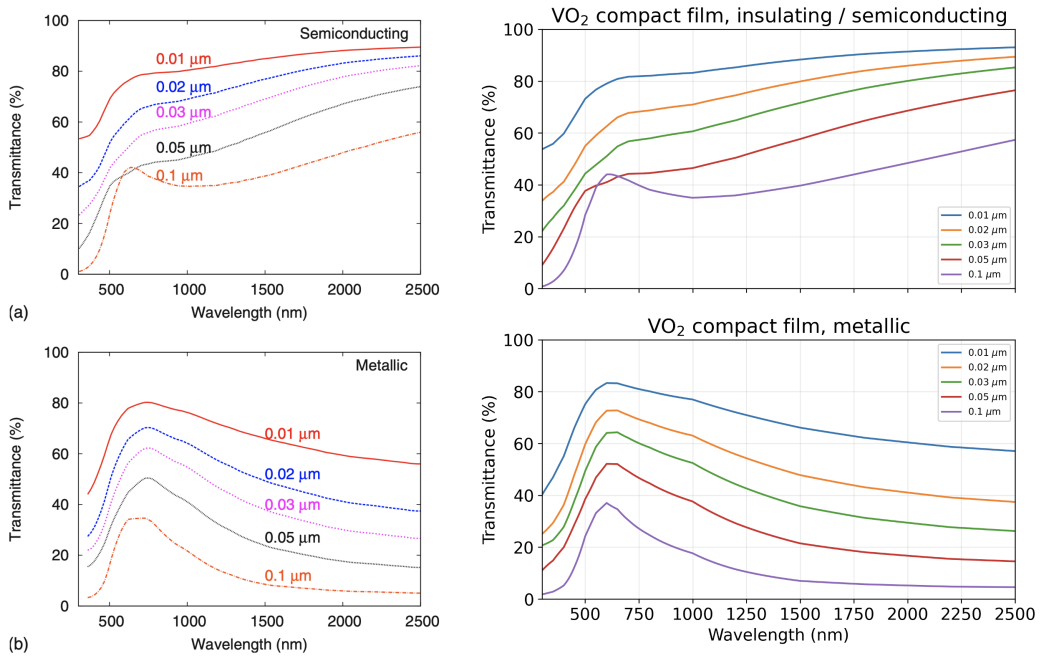


Figure 8: Spectral transmittance of compact VO₂ films in insulating and metallic states. Left: reproduced from Li et al. [\[10\]](#). Right: `nanochromic.py`.

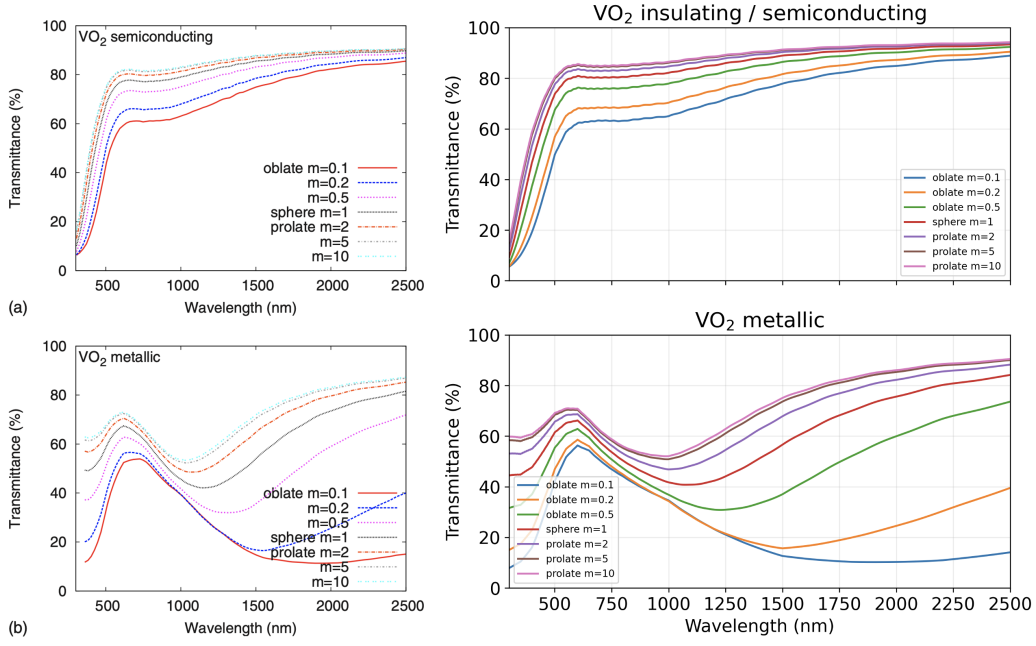


Figure 9: Spectral transmittance of oriented spheroidal VO_2 particles with $f = 0.01$. Left: reproduced from Li et al. [10]. Right: `nanochromic.py`. The symmetry axes are normal to the substrate, so the transverse depolarization factor is used.

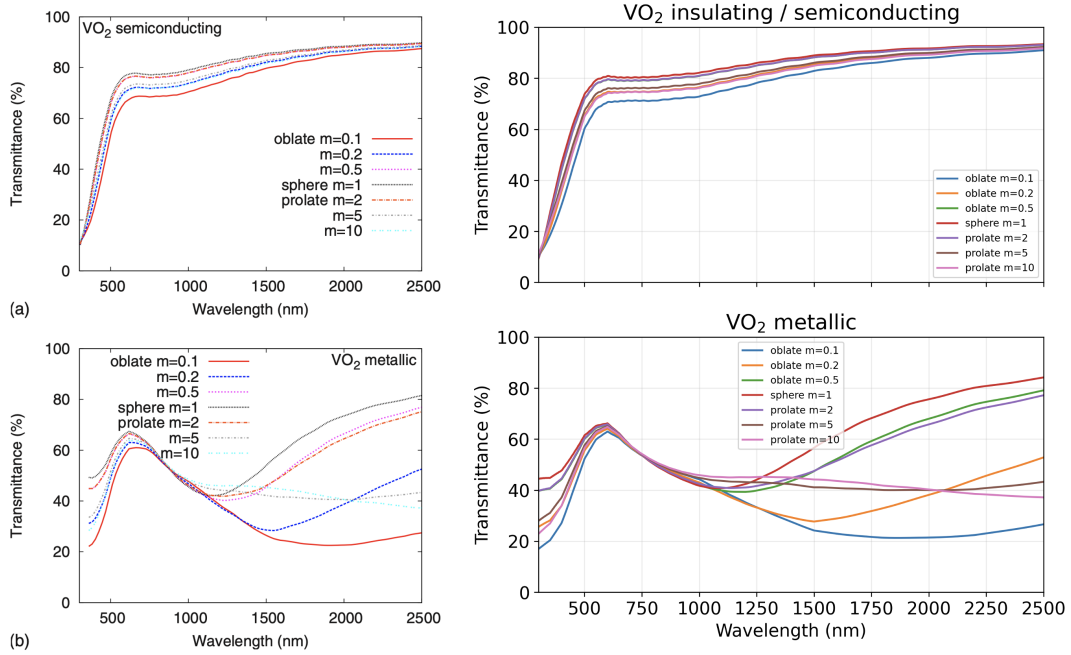


Figure 10: Spectral transmittance of randomly oriented spheroidal VO_2 particles with $f = 0.01$. Left: reproduced from Li et al. [10]. Right: `nanochromic.py`.

The implementation reproduces the spectral features in figures 8 to 10, including the shape-dependent metallic resonance and the oriented/random contrast. Optical data reconstructed from published figures leave small numerical differences, but spectral shapes and trends agree.

3.5 Generalization of ChromaSolver

3.5.1 Main corrections and implementation choices

The original particle routine used a fixed PET matrix and a manual depolarization factor. The revised routine makes these inputs explicit:

$$(f, m, \text{orientation}, \varepsilon_m, \text{optical data set}).$$

It also enforces the identities

$$L_{\parallel} + 2L_{\perp} = 1, \quad f = 0 \implies \varepsilon_{\text{eff}} = \varepsilon_m, \quad \varepsilon_p = \varepsilon_m \implies \varepsilon_{\text{eff}} = \varepsilon_m, \quad \text{Im}[\tilde{n}_{\text{eff}}(\lambda)] \geq 0.$$

The code checks $0 \leq f < 1$, $m > 0$ when used, and $0 < L_j < 1$ for each selected depolarization factor. Algorithm 1 summarizes the implementation.

Algorithm 1 Effective optical constants of a VO₂ nanocomposite.

Require: Filling factor f , aspect ratio m (or a user-supplied L_{user} in direct mode), orientation, matrix dielectric function $\varepsilon_m(\lambda)$, and particle optical data $n_p^\sigma(\lambda), k_p^\sigma(\lambda)$

Ensure: Effective indices $\tilde{n}_{\text{eff}}^\sigma(\lambda)$ for $\sigma \in \{\text{ins}, \text{met}\}$

```

1: if direct mode is selected then
2:    $\mathcal{L} \leftarrow [L_{\text{user}}]$ 
3: else
4:   Compute  $(L_{\parallel}, L_{\perp}, L_{\perp})$  from  $m$ 
5:   if orientation is random then
6:      $\mathcal{L} \leftarrow [L_{\parallel}, L_{\perp}, L_{\perp}]$ 
7:   else if the symmetry-axis response is selected then
8:      $\mathcal{L} \leftarrow [L_{\parallel}]$ 
9:   else
10:     $\mathcal{L} \leftarrow [L_{\perp}]$ 
11:   end if
12: end if
13: for  $\sigma \in \{\text{ins}, \text{met}\}$  do
14:    $\varepsilon_p^\sigma(\lambda) \leftarrow [n_p^\sigma(\lambda) + ik_p^\sigma(\lambda)]^2$ 
15:   for  $L_j \in \mathcal{L}$  do
16:      $\alpha_j(\lambda) \leftarrow \frac{\varepsilon_p^\sigma(\lambda) - \varepsilon_m(\lambda)}{\varepsilon_m(\lambda) + L_j [\varepsilon_p^\sigma(\lambda) - \varepsilon_m(\lambda)]}$ 
17:   end for
18:    $\bar{\alpha}(\lambda) \leftarrow \frac{1}{|\mathcal{L}|} \sum_{L_j \in \mathcal{L}} \alpha_j(\lambda)$ 
19:    $\varepsilon_{\text{eff}}^\sigma(\lambda) \leftarrow \varepsilon_m(\lambda) \frac{1 + \frac{2}{3}f\bar{\alpha}(\lambda)}{1 - \frac{1}{3}f\bar{\alpha}(\lambda)}$ 
20:   Choose  $\tilde{n}_{\text{eff}}^\sigma(\lambda)$  pointwise as the square root of  $\varepsilon_{\text{eff}}^\sigma(\lambda)$  satisfying  $\text{Im}[\tilde{n}_{\text{eff}}^\sigma(\lambda)] \geq 0$ 
21: end for
```

Two implementation details matter. First, a NumPy assignment such as

```
e_comp = e_matrix; e_comp *= factor
```

can modify `e_matrix` in place because both names may reference one array, contaminating the metallic-state calculation. The corrected code evaluates Maxwell–Garnett directly:

```

1 epsilon_eff = epsilon_m * (
2     1.0 + (2.0 / 3.0) * f * alpha
3 ) / (
4     1.0 - (1.0 / 3.0) * f * alpha
5 )

```

This leaves ϵ_m unchanged. Second, np.sqrt selects one complex square-root branch, whereas passivity requires $\text{Im}[\tilde{n}_{\text{eff}}(\lambda)] \geq 0$. The sign is therefore corrected when necessary:

```

1 root = np.sqrt(np.asarray(epsilon_eff, dtype=np.complex128))
2 n_eff = np.where(np.imag(root) < 0.0, -root, root)

```

The original PET option is retained for backward compatibility. The Li–Niklasson–Granqvist configuration instead fixes $n_m = 1.5$, $k_m = 0$ and $\epsilon_m := (n_m + ik_m)^2 = 2.25$.

3.5.2 Validation with integrated metrics

The modified solver was compared with Li et al.’s integrated curves for compact films and oriented or random spheroids.

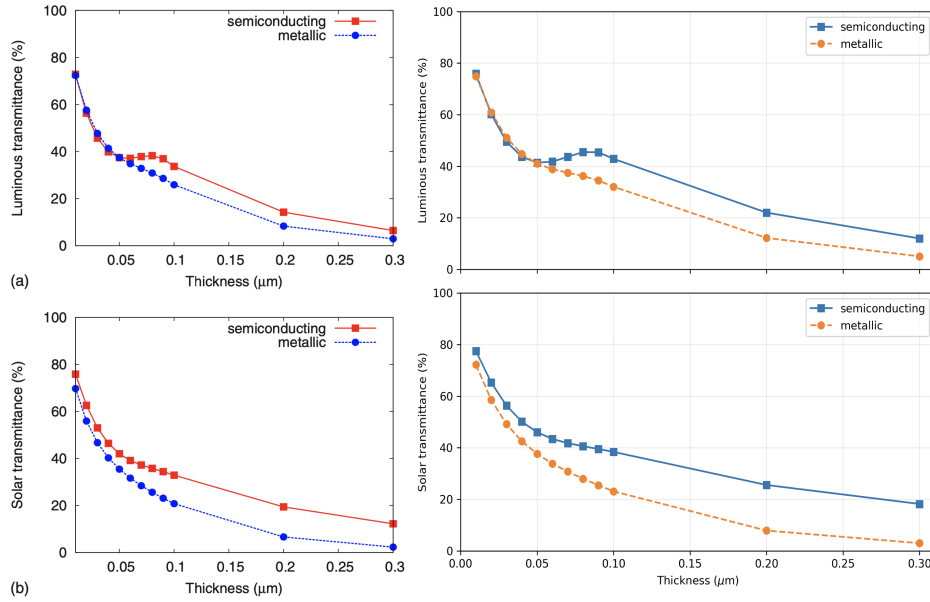


Figure 11: Luminous and solar transmittance of compact VO_2 films as functions of thickness. Left: reproduced from Li et al. [10]. Right: modified ChromaSolver.

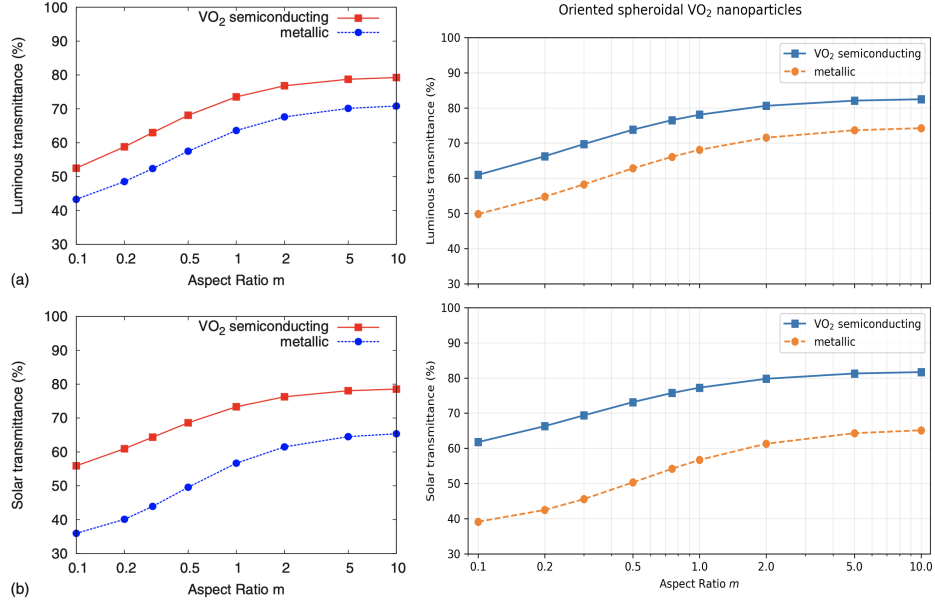


Figure 12: Luminous and solar transmittance of oriented spheroidal particles as functions of aspect ratio m , with $f = 0.01$. Left: reproduced from Li et al. [10]. Right: modified ChromaSolver.

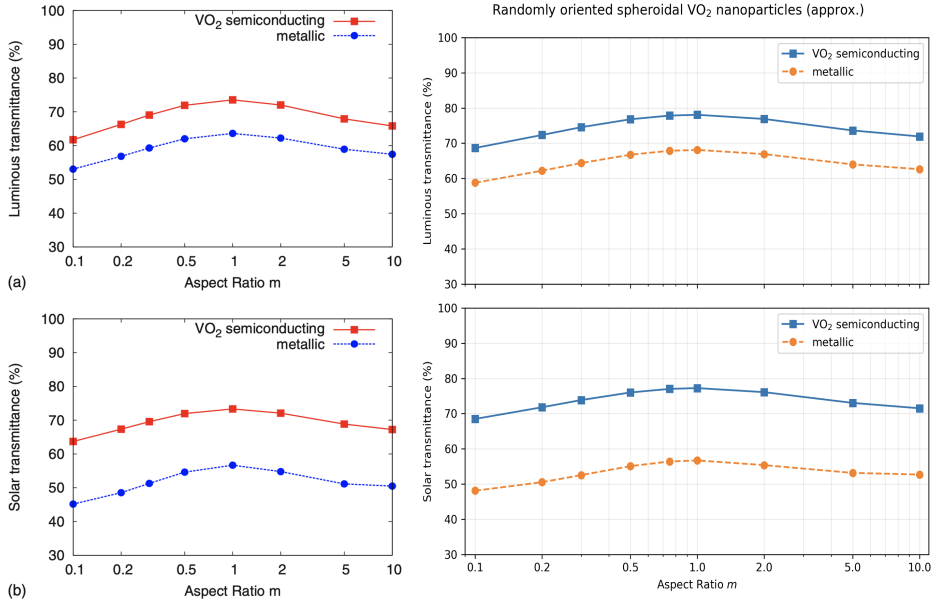


Figure 13: Luminous and solar transmittance of randomly oriented spheroidal particles as functions of aspect ratio m , with $f = 0.01$. Left: reproduced from Li et al. [10]. Right: modified ChromaSolver.

For compact films, both calculations show rapidly decreasing T_{lum} and T_{sol} with thickness, including the weak insulating-state interference feature. ChromaSolver is systematically higher: near $d = 0.20 \mu\text{m}$, the reference gives about 14% and 8% for insulating and metallic luminous transmittance, versus 22% and 12%. For oriented particles, both predict increasing transmittance from oblate to prolate shapes; near $m = 1$, ChromaSolver gives

$$T_{\text{sol}}^{\text{cold}} \simeq 0.77, \quad T_{\text{sol}}^{\text{hot}} \simeq 0.56\text{--}0.57, \quad \Delta T_{\text{sol}} \simeq 0.20.$$

Thus a dilute nanoparticle composite yields much larger solar modulation than a compact film with the same VO_2 volume per unit area. For random orientations, both calculations have a broad maximum near $m = 1$; reproducing it requires the polarizability average in [equation \(5\)](#), not prior averaging of

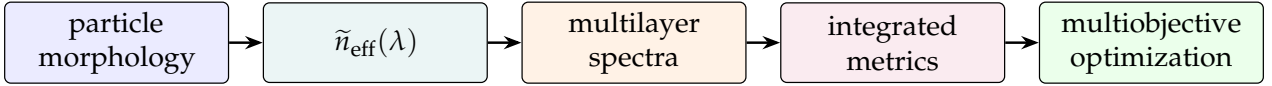
the L_j . A quantitative comparison would require digitized reference values y_j^{ref} and, for each metric,

$$E_\infty := \max_j |y_j^{\text{code}} - y_j^{\text{ref}}| \quad \text{and} \quad E_2 := \left(\frac{1}{J} \sum_{j=1}^J |y_j^{\text{code}} - y_j^{\text{ref}}|^2 \right)^{1/2}.$$

No common machine-readable reference data set was available, so these errors were not computed and the offsets remain unattributed.

4 Conclusion

This internship established a direct modeling chain from particle morphology to multilayer optimization:



The transfer-matrix workflow identified an optimum geometry differing from the published $\text{TiO}_2/\text{VO}_2/\text{TiO}_2$ optimum by at most 2 nm in each optimized thickness. Geometry and interference trends agree, but absolute luminous transmittance still requires better material and interface calibration.

The Li–Niklasson–Granqvist equations were implemented independently and integrated into Chroma-Solver with a configurable matrix, one aspect-ratio convention, the transverse response for vertically oriented spheroids at normal incidence, and principal-axis polarizability averaging for random orientations. The implementation also corrects in-place matrix modification and the complex square-root branch. It reproduces the main compact-film and spheroid trends, including about 20% solar modulation near spherical particles at $f = 0.01$, although absolute-transmittance offsets remain.

The next step is to optimize over the mixed design space $\mathcal{X}_{\text{mix}} := \mathcal{X}_{\text{cont}} \times \mathcal{X}_{\text{disc}}$ with

$$\min_{\mathbf{x} \in \mathcal{X}_{\text{mix}}} \left(-T_{\text{lum}}^{\text{min}}(\mathbf{x}), -\Delta T_{\text{sol}}(\mathbf{x}), \max_{\tau \in \{\text{cold}, \text{hot}\}} \bar{\varepsilon}_{\text{LWIR}}^{\tau}(\mathbf{x}) \right),$$

where the third objective is the worst-state LWIR emissivity and $\mathbf{x}$ combines layer- and particle-scale variables, for example

$$\mathbf{x} = (d, f, m, \eta, \chi_m),$$

with orientation choice η and dielectric-matrix choice χ_m .

A Detailed software architecture and execution workflow

A.1 Purpose and scope

This appendix records the software path needed to reproduce a run: modules, configuration, calls, interpolation, parallel evaluation, serialization, post-processing, failures, and outputs. It does not repeat the physics of [sections 1, 2.1 and 3.3](#). The examples use one representative optimizer configuration; each experiment’s states, materials, wavelength grid, and budget remain configuration-dependent. Genetic algorithms and NSGA-II are summarized in [section B](#).

A.2 Modules and data products

The modules in [figure 6](#) are mapped in [table 3](#): the first four form the simulation–optimization loop, and the others analyze or independently check saved results.

Table 3: Python modules used in the modeling and optimization workflow.

Module	Primary responsibility	Principal output
Parallel.py	Defines the design variables, objectives, constraints, and NSGA-II operators; evaluates candidate geometries; converts physical metrics into optimizer arrays; records the complete search history.	Population data and final CSV history.
simulation_angles_pols.py	Builds the optical stacks, selects coherent or mixed coherent-incoherent propagation, loops over incidence angles and polarizations, and dispatches spectral calculations.	Per-angle and per-polarization result dictionary; full_data.npz.
chromasolver.py	Loads and interpolates optical data, constructs wavelength-dependent complex indices, calls the tmm library, and returns spectral and integrated quantities for one stack.	Spectra, target-layer absorption, absorbed power, T_{lum} , T_{sol} , and transmission at 550 nm.
postprocessing_angles_pols.py	Loads saved spectra, combines polarizations and angles, evaluates LWIR emissivity and thermochromic differences, and assembles the metric dictionary used by the optimizer.	Scalar metric dictionary.
GA_Results_Plot.py	Reads the optimization CSV and displays maps or scatter plots in the design-variable space.	Metric maps and diagnostic plots.
StacksComparison.py	Loads selected full_data.npz files and compares spectra and aggregate metrics between stacks.	Spectral comparison and radar plots.
nanochromic.py	Implements the nanothermochromic effective-medium model independently of the optimization framework, thereby reducing the number of possible error sources during validation.	Stand-alone compact-film and nanocomposite spectra.

A.3 Representative optimization configuration

A.3.1 Thickness template and bounds

An NSGA-II candidate stores only variable thicknesses; fixed entries remain in a template and None marks variables. The representative diagnostic configuration is

```

1 base_t = [None, None, 190, 210, 200, 1000, 200]
2 bounds = [(25, 200), (25, 200)]
3
4 pop = 32
5 gen = 2

```

The two optimization variables are therefore bounded by

$$25 \text{ nm} \leq x_1 \leq 200 \text{ nm}, \quad 25 \text{ nm} \leq x_2 \leq 200 \text{ nm}.$$

The other thicknesses are fixed at (190, 210, 200, 1000, 200) nm; two generations are sufficient for an execution test.

A.3.2 Objectives and constraints

Dictionary lists define objectives and constraints without changing the evaluation routine. One example is

```

1 Tlum_tar = 40.0
2
3 objectives = [
4     {"metric": "Tlum_met", "type": "max"},
5 ]

```

```

6
7 constraints = [
8     {"metric": "Tlum_met", "operator": ">",
9      "threshold": Tlum_tar},
10 ]

```

Since pymoo minimizes by convention, maximizing Tlum_met uses $F := -\text{Tlum_met}$. The visible-transmittance condition is

$$G := 40 - \text{Tlum_met},$$

which is feasible when $G \leq 0$. The same conversion rules are used for any metric name present in the post-processing dictionary.

A.3.3 Optical metadata and state semantics

Metadata specify materials, coherence, angle and wavelength grids, polarizations, outputs, and the relation between optical states. With explicit Boolean values,

```

1 metadata = {
2     "name_met": [
3         "SiO2_thin", "TiO2_FB", "Al2O3",
4         "Si3N4_DSRCG", "ITO", "glass", "ITO"
5     ],
6     "name_ins": [
7         "SiO2_thin", "TiO2_FB", "Al2O3",
8         "Si3N4_DSRCG", "ITO", "glass", "ITO"
9     ],
10    "c_list": [True, True, True, True, True, False, True],
11    "thetas": [0],
12    "wlsIR": np.arange(380, 13000, 1),
13    "pols": ["s", "p"],
14    "save": True,
15    "dual_stack": False,
16 }

```

The grid spans integer wavelengths from 380 nm to 12 999 nm; glass is incoherent, thinner layers are coherent, and both polarizations are evaluated at normal incidence. Because name_met and name_ins coincide and dual_stack=False, one result populates both formal states. Hence delta_Tsol, delta_e, and the FOM are valid outputs but not physical thermochromic modulations; those require distinct state data or stacks.

A.3.4 NSGA-II operators and termination

The optimizer uses random real sampling, simulated binary crossover, polynomial mutation, and duplicate elimination:

```

1 algo = NSGA2(
2     pop_size=pop,
3     sampling=FloatRandomSampling(),
4     crossover=SBX(prob=0.8, eta=5),
5     mutation=PM(prob=0.4, eta=5),
6     eliminate_duplicates=True,
7 )
8
9 res = minimize(
10    prob,
11    algo,
12    get_termination("n_gen", gen),
13    seed=42,
14    verbose=True,
15    save_history=True,

```

The seed belongs to the run definition because it controls initialization and stochastic operators.

A.4 Candidate representation and population evaluation

A.4.1 Detection and reconstruction of variable thicknesses

Variable-layer positions are extracted from the template:

```
1 arr = np.array(base_t, dtype=object)
2 variable_indices = np.where(arr == None)[0]
```

Here `variable_indices` is `[0, 1]`; these indices, bounds, template, objectives, and constraints define the problem:

```
1 prob = V02StackProblem(
2     variable_indices=variable_indices,
3     bounds=bounds,
4     fixed_thicknesses=base_t,
5     objective_config=objectives,
6     constraint_config=constraints,
7 )
```

In `_evaluate(self, X, out, ...)`, each row of `X` is one variable-thickness vector; the full geometry is rebuilt from a template copy:

```
1 t_full = fixed_t.copy()
2 for i, idx in enumerate(var_idx):
3     t_full[idx] = x[i]
```

For example, $x = (72.4, 143.8)$ produces

$$d_{\text{full}} = (72.4, 143.8, 190, 210, 200, 1000, 200) \text{ nm.}$$

A.4.2 Independent evaluation and multiprocessing

Each individual's arguments are assembled before dispatch:

```
1 args_list = [
2     (x, self.variable_indices, self.fixed_thicknesses,
3     self.objective_config, self.constraint_config,
4     False, self.metadata)
5     for x in X
6 ]
```

Locally, the implementation leaves one core free and evaluates individuals in a process pool:

```
1 nproc = multiprocessing.cpu_count() - 1
2 with Pool(processes=nproc) as pool:
3     results = pool.map(eval_one_ind, args_list)
```

Each independent `eval_one_ind` call rebuilds a thickness vector, runs optical simulation and post-processing, and returns objectives, constraints, metrics, geometry, and feasibility. Ordered `pool.map` preserves alignment with `X`.

A.5 Stack construction and coherent–incoherent dispatch

The full thickness vector is sent to

```
1 stack_dir, stack_name = simu.simu_stack(
```



```

2     metadata["name_met"],
3     metadata["name_ins"],
4     t_full,
5     metadata["c_list"],
6     metadata["thetas"],
7     metadata["wlsIR"],
8     metadata["pols"],
9     metadata["save"],
10    metadata["dual_stack"],
11 )

```

Within `simulation_angles_pols.py`, each finite layer is represented by

(material name, thickness, coherence flag),

and semi-infinite air media are added on both sides. In schematic form,

```

1 stack_met = [("Air", np.inf, False)]
2 stack_ins = [("Air", np.inf, False)]
3
4 for i in range(len(name_met)):
5     stack_met.append((name_met[i], t[i], c_list[i]))
6     stack_ins.append((name_ins[i], t[i], c_list[i]))
7
8 stack_met.append(("Air", np.inf, False))
9 stack_ins.append(("Air", np.inf, False))

```

Layer flags select coherence: fully coherent stacks use the coherent solver; otherwise the mixed solver retains thin-film interference without imposing it on thick substrates:

```

1 coherence = True
2 for c in c_list:
3     if c is False:
4         coherence = False
5         break

```

This Boolean selects `coh_tmm` or `inc_tmm`, which also receives the coherence list.

A.6 Optical input data and preprocessing

A.6.1 Input files

Initialize the optical simulation by

```

1 sim = cs.Simulation(wlsIR, data_file, thermo=True)

```

The configuration uses two principal tables:

Table 4: Principal tabulated inputs used by `chromasolver.py`.

File	Contents and use
Opti_data.csv	Wavelength-dependent material constants $n(\lambda)$ and $k(\lambda)$, including the photopic-sensitivity column used for T_{lum} .
SolarAM15.csv	AM1.5 spectral irradiance used for T_{sol} and for the target-layer absorbed-power calculation.

Before the transfer-matrix loop, the tables are read and mapped to the requested grid:

```

1 self.nkdata = pd.read_csv(nkdata_path, sep=";", decimal=",")
2 self.power = pd.read_csv(solar_file, header=0, sep=";")
3 self.nkdata, self.power = self.interpolate_data(

```

```

4     self.wavelengths,
5     self.nkdata,
6     self.power,
7 )

```

A.6.2 Interpolation and precomputed arrays

Interpolation is linear and allows extrapolation:

```

1 interpolator = interp1d(
2     nkdata[columns[index]],
3     nkdata[columns[index + 1]],
4     kind="linear",
5     fill_value="extrapolate",
6 )
7 interpolated_df[columns[index + 1]] = interpolator(
8     interpolated_df["used_wls"]
9 )

```

The tabulated range must therefore cover the requested grid whenever physical extrapolation is not justified; outside it, optical constants are only numerical continuations. Wavelength, photopic, solar-power, and complex-index arrays are precomputed to avoid repeated DataFrame access:

```

1 wls_arr = self.nkdata["used_wls"].to_numpy(dtype=float)
2 eye_arr = self.nkdata["Eye_sensitivity"].to_numpy(dtype=float)
3 power_arr = self.power[power_column].to_numpy(dtype=float)
4
5 layer_n_complex = []
6 for material, thickness, coherent in real_layers:
7     n = self.nkdata[material + "_n"].to_numpy(dtype=float)
8     k = self.nkdata[material + "_k"].to_numpy(dtype=float)
9     layer_n_complex.append(n + 1j * k)

```

At wavelength index index, direct indexing gives the refractive-index list:

```

1 refr_indices = [arr[index] for arr in layer_n_complex]

```

For a nanocomposite, [section 3.5.1](#) replaces the selected compact index by $\tilde{n}_{\text{eff}}(\lambda)$; all other stack logic is unchanged.

A.7 Spectral calculation and returned quantities

A.7.1 Angles, polarizations, and target layer

The dispatcher loops over requested angles and polarizations:

```

1 for theta in thetas:
2     theta_rad = np.deg2rad(theta)
3     results[theta] = {}
4     for pol in pols:
5         res_single = sim.sim_stack(
6             stack_single,
7             index_of_vo2_layer=2,
8             coherent=coherence,
9             THETA=theta_rad,
10            POL=pol,
11        )

```

For an isotropic stack at normal incidence, s and p should coincide; both branches are retained for oblique incidence. The historical name `index_of_vo2_layer` only marks the separately stored absorption layer and denotes VO₂ only when appropriate.

A.7.2 Transfer-matrix dispatch and layer-resolved absorption

At each wavelength, the coherence mode selects the solver:

```
1 if coherent:
2     tmm_data = tmm.coh_tmm(
3         POL, refr_indices, thicknesses, THETA, wl
4     )
5     absorb_per_layer_data = tmm.absorp_in_each_layer(tmm_data)
6 else:
7     tmm_data = tmm.inc_tmm(
8         POL, refr_indices, thicknesses,
9         coherence_list, THETA, wl
10    )
11    absorb_per_layer_data = tmm.inc_absorp_in_each_layer(tmm_data)
```

The per-layer result yields total reflection, total transmission, and target-layer absorption:

```
1 abs_in_vo2[index] = absorb_per_layer_data[index_of_vo2_layer]
2 tot_trans[index] = absorb_per_layer_data[-1]
3 tot_refl[index] = absorb_per_layer_data[0]
```

AM1.5 weights target-layer absorbed solar power; the factor 10 converts mW/cm² to W/m²:

```
1 P_abs_tot += (
2     10.0
3     * absorb_per_layer_data[index_of_vo2_layer]
4     * power_arr[index]
5 )
```

Visible and solar transmittances use trapezoidal quadrature:

```
1 Tlum = np.trapezoid(T_visible * V_visible, wl_visible) \
2     / np.trapezoid(V_visible, wl_visible)
3
4 Tsol = np.trapezoid(T_solar * P_AM15, wl_solar) \
5     / np.trapezoid(P_AM15, wl_solar)
```

The function returns percentage-valued spectra, target-layer absorption, integrated transmittances, and transmission at 550 nm:

```
1 return (
2     100 * tot_trans,
3     100 * tot_refl,
4     100 * abs_in_vo2,
5     P_abs_tot,
6     100 * Tlum,
7     100 * Tsol,
8     100 * trans550,
9 )
```

This is an implementation convention: the main text uses fractions in $[0, 1]$, and post-processing divides percentages by 100 when needed.

A.8 Serialization and post-processing

A.8.1 Saved spectral data

For each angle and polarization, the dispatcher stores spectra and integrated quantities under explicit state labels:

```
1 results[theta][pol] = {
2     "E_met": E_met,
3     "E_ins": E_ins,
```

```

4     "T_met": T_met,
5     "T_ins": T_ins,
6     "R_met": R_met,
7     "R_ins": R_ins,
8     "Tlum_met": Tlum_met,
9     "Tlum_ins": Tlum_ins,
10    "Tsol_met": Tsol_met,
11    "Tsol_ins": Tsol_ins,
12 }

```

Here E denotes spectral emissivity or absorptance $1 - T/100 - R/100$, not an electric field. The record is serialized as

```

1 np.savez(
2     os.path.join(save_dir, "full_data.npz"),
3     wavelengths=wl,
4     thetas=thetas,
5     results=results,
6     stack_met=stack_met,
7     stack_ins=stack_ins,
8 )

```

Post-processing reloads the file with

```

1 data = np.load(data_file, allow_pickle=True)
2 wl = data["wavelengths"]
3 results = data["results"].item()
4 thetas = data["thetas"]

```

Because the nested dictionary is a Python object, `allow_pickle=True` should be used only for trusted project files.

A.8.2 Polarization combination

The documented value `pol=0` corresponds to an equal-weight average of the two polarizations:

$$Q := \frac{1}{2}Q_s + \frac{1}{2}Q_p.$$

For example,

```

1 s = 0.5
2 p = 0.5
3 T_met_theta.append(
4     s * results[theta]["s"]["T_met"]
5     + p * results[theta]["p"]["T_met"]
6 )

```

At normal incidence this average is redundant for isotropic layers but preserves the angle-dependent data structure.

A.8.3 LWIR integration and final metric dictionary

Planck weighting is evaluated at $T_{\text{BB}} := 300$ K and integrated by trapezoidal quadrature on 8–13 μm :

```

1 def planck_lambda(wavelength_m):
2     return (2 * h * c**2) / (wavelength_m**5) / (
3         np.exp(h * c / (wavelength_m * k * T_BB)) - 1
4     )
5
6 denom = np.trapezoid(I_BB, wl_LWIR)
7 num_met = np.trapezoid(E_met_spec * I_BB, wl_LWIR)
8 num_ins = np.trapezoid(E_ins_spec * I_BB, wl_LWIR)

```

```

9 e_LWIR_met.append(num_met / denom)
10 e_LWIR_ins.append(num_ins / denom)

```

The factor π in M_{BB} cancels in the normalized ratio, so its omission from `planck_lambda` does not change $\bar{\epsilon}_{\text{LWIR}}$.

The final dictionary returned to the optimizer is

```

1 return {
2     "Tlum_met": Tlum_met,
3     "Tlum_ins": Tlum_ins,
4     "Tsol_met": Tsol_met,
5     "Tsol_ins": Tsol_ins,
6     "delta_Tsol": delta_Tsol,
7     "e_LWIR_met": e_LWIR_met[0],
8     "e_LWIR_ins": e_LWIR_ins[0],
9     "delta_e": delta_LWIR,
10    "FOM": FOM,
11 }

```

Meanings and signs follow [section 1](#); physical modulation requires distinct state definitions.

A.9 Conversion to optimizer arrays and failure handling

The scalar dictionary becomes F by negating metrics marked for maximization:

```

1 f = []
2 for obj in obj_conf:
3     value = metrics[obj["metric"]]
4     f.append(-value if obj.get("type") == "max" else value)

```

Inequalities are mapped to the pymoo convention $G_j \leq 0$:

```

1 g = []
2 for constraint in constr_conf:
3     value = metrics[constraint["metric"]]
4     threshold = constraint["threshold"]
5     if constraint["operator"] in [ ">", ">=" ]:
6         g.append(threshold - value)
7     else:
8         g.append(value - threshold)

```

A failed simulation receives a large penalty and is marked infeasible:

```

1 except Exception as error:
2     return {
3         "F": [1e6] * len(obj_conf),
4         "G": [1e6] * len(constr_conf) if constr_conf else [],
5         "metrics": {},
6         "t_full": t_full,
7         "feasible": False,
8     }

```

A.10 Persistent history, analysis tools, and run-control flags

Each evaluation stores variables, full thicknesses, metrics, objectives, and constraints:

```

1 self.history_data.append({
2     "x": X[i].copy(),
3     "t_full": t_full_list[i],
4     "metrics": metrics_list[i],
5     "F": F_list[i],
6     "G": G_list[i] if self.n_ieq_constr > 0 else [],

```

At termination, the records are tabulated and written to CSV:

```
1 df = pd.DataFrame(records)
2 df.to_csv(save_path, index=False)
```

The CSV is the optimization history read by `GA_Results_Plot.py`; selected candidates also store `full_data.npz` for `StacksComparison.py`. The switches in [table 5](#) alter execution or plotting, not the physics.

Table 5: Run-control flags described in the code note.

Flag	Effect
PrivatePC, CECI	Select environment-dependent file paths and, in some versions, the numerical-integration implementation used on a local workstation or the CECI computing environment.
Para	Activates multiprocessing when <code>simulation_angles_pols.py</code> is executed directly.
emissivity	Enables construction of $E(\lambda) := 1 - T(\lambda)/100 - R(\lambda)/100$.
plot_transmission, plot_emissivity, LWIR_plot	Control the production of spectral and LWIR diagnostic figures.
hemi_T, hemi_E	Enable hemispherical quantities when a sufficiently resolved set of incidence angles is available.
dual_stack	Selects whether the two formal optical states are simulated as distinct stacks or share one calculated result.

B Genetic algorithms and NSGA-II

B.1 Optimization problem and population

Genetic algorithms are stochastic population methods [11, 12, 13, 14]. Here an individual is a real vector of layer or particle parameters,

$$\mathbf{x} := (x_1, \dots, x_{n_{\text{var}}}) \in \mathcal{X} := \prod_{j=1}^{n_{\text{var}}} [\ell_j, u_j] \subset \mathbb{R}^{n_{\text{var}}},$$

where $\ell_j := x_j^{\min}$ and $u_j := x_j^{\max}$ are the lower and upper bounds of variable x_j . For one objective,

$$\mathbf{x}^* \in \arg \max_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \iff \mathbf{x}^* \in \arg \min_{\mathbf{x} \in \mathcal{X}} [-f(\mathbf{x})].$$

For M objectives to minimize, define

$$\mathbf{F} : \mathcal{X} \longrightarrow \mathbb{R}^M, \quad \mathbf{F}(\mathbf{x}) := (f_1(\mathbf{x}), \dots, f_M(\mathbf{x})).$$

With inequality constraints $g_j(\mathbf{x}) \leq 0$, $j = 1, \dots, n_g$, and equality constraints $h_k(\mathbf{x}) = 0$, $k = 1, \dots, n_h$, the feasible set is

$$\mathcal{X}_{\text{feas}} := \{\mathbf{x} \in \mathcal{X} : g_j(\mathbf{x}) \leq 0 \ \forall j, \quad h_k(\mathbf{x}) = 0 \ \forall k\}.$$

The optimization problem is therefore $\min_{\mathbf{x} \in \mathcal{X}_{\text{feas}}} \mathbf{F}(\mathbf{x})$. At generation t , a population of size N_{pop} is

$$P_t := (\mathbf{x}_t^{(1)}, \dots, \mathbf{x}_t^{(N_{\text{pop}})}) \in \mathcal{X}^{N_{\text{pop}}}.$$

Population search explores several design-space regions simultaneously.

B.2 Canonical generation

One generation can be written as

$$A_t := \mathcal{S}_{\text{par}}(P_t), \quad B_t := \mathcal{C}(A_t), \quad Q_t := \mathcal{M}(B_t) \quad \text{and} \quad P_{t+1} := \mathcal{S}_{\text{surv}}(P_t, Q_t),$$

where $\mathcal{S}_{\text{par}}$, $\mathcal{C}$, $\mathcal{M}$, and $\mathcal{S}_{\text{surv}}$ denote parent selection, crossover, mutation, and survival selection. The optical model evaluates Q_t before survival.

For real vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^{n_{\text{var}}}$, arithmetic crossover is

$$\mathbf{z}_1 := \alpha \mathbf{x} + (1 - \alpha) \mathbf{y} \quad \text{and} \quad \mathbf{z}_2 := (1 - \alpha) \mathbf{x} + \alpha \mathbf{y}, \quad \alpha \in [0, 1].$$

A simple mutation is

$$\mathbf{x}' := \mathbf{x} + \sigma \mathbf{Z}, \quad \text{with} \quad \mathbf{Z} \sim \mathcal{N}(0, I_{n_{\text{var}}}) \quad \text{and} \quad \sigma > 0.$$

Bounds may use repair, reflection, resampling, or clipping. The implemented operators are simulated binary crossover and polynomial mutation.

B.2.1 Simulated binary crossover

For parents $\mathbf{x}^{(1)}$ and $\mathbf{x}^{(2)}$, the unbounded core SBX map acts coordinatewise. Draw $u \sim \mathcal{U}(0, 1)$ and set

$$\beta(u) := \begin{cases} (2u)^{1/(\eta_c+1)}, & 0 \leq u \leq \frac{1}{2}, \\ \left[\frac{1}{2(1-u)} \right]^{1/(\eta_c+1)}, & \frac{1}{2} < u \leq 1, \end{cases}$$

where $\eta_c > 0$ is the crossover distribution index. The two child coordinates are

$$\begin{aligned} z_j^{(1)} &:= \frac{1}{2} \left[(1 + \beta) x_j^{(1)} + (1 - \beta) x_j^{(2)} \right], \\ z_j^{(2)} &:= \frac{1}{2} \left[(1 - \beta) x_j^{(1)} + (1 + \beta) x_j^{(2)} \right]. \end{aligned}$$

Their midpoint is the parent midpoint,

$$\frac{z_j^{(1)} + z_j^{(2)}}{2} = \frac{x_j^{(1)} + x_j^{(2)}}{2},$$

while their separation is scaled by β :

$$z_j^{(1)} - z_j^{(2)} = \beta \left(x_j^{(1)} - x_j^{(2)} \right).$$

Thus $\beta < 1$ interpolates and $\beta > 1$ extrapolates; larger η_c concentrates β near 1, keeping children close to their parents. Bounds are enforced separately. In the documented run, SBX(prob=0.8, eta=5) uses crossover probability 0.8 and $\eta_c = 5$; pymoo handles variable-level participation.

B.2.2 Polynomial mutation

For a coordinate $x_j \in [\ell_j, u_j]$, define its normalized distances to the lower and upper bounds,

$$\delta_1 := \frac{x_j - \ell_j}{u_j - \ell_j} \quad \text{and} \quad \delta_2 := \frac{u_j - x_j}{u_j - \ell_j}.$$

Draw $r \sim \mathcal{U}(0, 1)$, set $\gamma_m := 1/(\eta_m + 1)$, and define the normalized mutation step

$$\delta_q := \begin{cases} [2r + (1 - 2r)(1 - \delta_1)^{\eta_m+1}]^{\gamma_m} - 1, & r \leq \frac{1}{2}, \\ 1 - [2(1 - r) + 2(r - \frac{1}{2})(1 - \delta_2)^{\eta_m+1}]^{\gamma_m}, & r > \frac{1}{2}. \end{cases}$$

The mutated coordinate is

$$x'_j := x_j + \delta_q(u_j - \ell_j),$$

followed by bound handling. Larger η_m favors local changes; smaller values yield heavier tails. The run uses `PM(prob=0.4, eta=5)`, i.e. individual-level probability 0.4 and $\eta_m = 5$, with a separate variable-level probability in `pymoo`.

B.3 Multiobjective dominance

For M objectives to minimize,

$$F(\mathbf{x}) := (f_1(\mathbf{x}), \dots, f_M(\mathbf{x})).$$

A point $\mathbf{x}$ dominates $\mathbf{y}$, written $\mathbf{x} \prec \mathbf{y}$, if

$$[\forall r, f_r(\mathbf{x}) \leq f_r(\mathbf{y})] \quad \text{and} \quad [\exists r, f_r(\mathbf{x}) < f_r(\mathbf{y})].$$

Equivalently,

$$\mathbf{x} \prec \mathbf{y} \iff F(\mathbf{x}) \in F(\mathbf{y}) - \mathbb{R}_+^M \quad \text{and} \quad F(\mathbf{x}) \neq F(\mathbf{y}),$$

where

$$\mathbb{R}_+^M := \{ \mathbf{z} \in \mathbb{R}^M : z_r \geq 0 \forall r \}.$$

A feasible point is Pareto optimal if no other feasible point dominates it. The Pareto set and Pareto front are

$$\mathcal{P} := \{ \mathbf{x} \in \mathcal{X}_{\text{feas}} : \nexists \mathbf{y} \in \mathcal{X}_{\text{feas}}, \mathbf{y} \prec \mathbf{x} \} \quad \text{and} \quad \mathcal{F} := F(\mathcal{P}).$$

The goal is a diverse approximation of $\mathcal{F}$, not a single optimum. For a finite population R , the first non-dominated front is

$$F_1 := \{ \mathbf{x} \in R : \nexists \mathbf{y} \in R, \mathbf{y} \prec \mathbf{x} \}.$$

After removing F_1 , the second front is the non-dominated set of the remaining points,

$$F_2 := \{ \mathbf{x} \in R \setminus F_1 : \nexists \mathbf{y} \in R \setminus F_1, \mathbf{y} \prec \mathbf{x} \},$$

and recursively thereafter. The Pareto rank of $\mathbf{x}$ is

$$\text{rank}(\mathbf{x}) := k \iff \mathbf{x} \in F_k.$$

Smaller rank is preferred.

B.4 NSGA-II selection

NSGA-II combines non-dominated sorting, elitism, and crowding distance [15]. At generation t :

1. Merge parents and offspring: $R_t := P_t \cup Q_t$.
2. Split R_t into non-dominated fronts $F_1, F_2, \dots$
3. Add complete fronts until the next front would exceed size N_{pop} .
4. Fill the remaining places with the least crowded points of that front.

Crowding distance measures local objective-space spacing. For a front F_k of size N_k , sort each objective r so that

$$f_r(\mathbf{x}_{(1)}) \leq f_r(\mathbf{x}_{(2)}) \leq \dots \leq f_r(\mathbf{x}_{(N_k)}).$$

Boundary points receive infinite distance for that objective. For an interior point $\mathbf{x}_{(i)}$, $2 \leq i \leq N_k - 1$, the contribution is

$$d_r(\mathbf{x}_{(i)}) := \frac{f_r(\mathbf{x}_{(i+1)}) - f_r(\mathbf{x}_{(i-1)})}{f_r^{\max} - f_r^{\min}},$$

where

$$f_r^{\min} := \min_{\mathbf{x} \in F_k} f_r(\mathbf{x}) \quad \text{and} \quad f_r^{\max} := \max_{\mathbf{x} \in F_k} f_r(\mathbf{x}).$$

The total crowding distance is

$$d(\mathbf{x}) := \sum_{r=1}^M d_r(\mathbf{x}).$$

If $f_r^{\max} = f_r^{\min}$, objective r contributes zero; otherwise normalization by $f_r^{\max} - f_r^{\min}$ makes scales comparable. Large $d(\mathbf{x})$ means that $\mathbf{x}$ lies in a sparse front region. Rank and crowding define NSGA-II preference. For two candidates $\mathbf{x}$ and $\mathbf{y}$, write $\mathbf{x} \triangleleft \mathbf{y}$ when $\text{rank}(\mathbf{x}) < \text{rank}(\mathbf{y})$ or when

$$\text{rank}(\mathbf{x}) = \text{rank}(\mathbf{y}) \quad \text{and} \quad d(\mathbf{x}) > d(\mathbf{y}).$$

Thus rank precedes diversity. Building P_{t+1} from $R_t = P_t \cup Q_t$ is elitist: good parents may survive without reappearing in Q_t .

B.5 Constraints, budget, and stopping

For inequalities $g_j(\mathbf{x}) \leq 0$ and equalities $h_k(\mathbf{x}) = 0$, one possible penalty for a scalar minimization objective is

$$f_\rho(\mathbf{x}) := f(\mathbf{x}) + \rho \left[\sum_j \max(0, g_j(\mathbf{x}))^2 + \sum_k h_k(\mathbf{x})^2 \right], \quad \rho > 0.$$

A related measure of total constraint violation is

$$\text{CV}(\mathbf{x}) := \sum_j \max(0, g_j(\mathbf{x})) + \sum_k |h_k(\mathbf{x})|,$$

so $\mathbf{x} \in \mathcal{X}_{\text{feas}}$ iff $\text{CV}(\mathbf{x}) = 0$. Feasibility rules avoid choosing one penalty scale across objectives with different units. The code passes inequalities as $G_j(\mathbf{x}) \leq 0$. If the initial population has size N_{pop} , each later generation evaluates N_{off} new individuals, and G denotes the pymoo generation count used by `get_termination("n_gen", G)`, then

$$C_{\text{eval}} \simeq N_{\text{pop}} + (G - 1)N_{\text{off}}, \quad G \geq 1.$$

If one optical evaluation costs an average time c_{opt} , a serial run has the rough cost $C_{\text{time}} \simeq c_{\text{opt}} C_{\text{eval}}$, which motivates parallel evaluation. Stopping may use an evaluation budget, a target metric, or a fixed stagnation period. For real vectors, a simple diversity measure is

$$D_{\text{var}}(P) := \frac{1}{N_{\text{pop}}} \sum_{i=1}^{N_{\text{pop}}} \left\| \mathbf{x}^{(i)} - \bar{\mathbf{x}} \right\|^2 \quad \text{and} \quad \bar{\mathbf{x}} := \frac{1}{N_{\text{pop}}} \sum_{i=1}^{N_{\text{pop}}} \mathbf{x}^{(i)}.$$

This is the mean squared distance to the population centroid; expanding gives

$$D_{\text{var}}(P) = \sum_{j=1}^{n_{\text{var}}} \left[\frac{1}{N_{\text{pop}}} \sum_{i=1}^{N_{\text{pop}}} \left(x_j^{(i)} - \bar{x}_j \right)^2 \right],$$

Thus D_{var} sums coordinatewise variances; small values indicate population collapse. Stronger mutation, weaker selection, random immigrants, or restarts increase diversity. For variables with different units or ranges, use

$$\hat{x}_j := \frac{x_j - \ell_j}{u_j - \ell_j} \in [0, 1]$$

in the same formula, separating loss of diversity from scale differences.

References

- [1] Shancheng Wang et al. “Scalable thermochromic smart windows with passive radiative cooling regulation”. In: *Science* 374.6574 (2021), pp. 1501–1504. DOI: [10.1126/science.abg0291](https://doi.org/10.1126/science.abg0291).
- [2] Jaroslav Vlček, Michal Kaufman, Elnaz Mohammadi Nia, Jiří Houška, Jiechao Jiang, Radomír Čerstvý, Stanislav Haviar, and Efstathios I. Meletis. “High-performance thermochromic multilayer coatings of W-doped VO₂ nanoparticles dispersed in an SiO₂ matrix prepared on glass at a low temperature”. In: *ACS Applied Nano Materials* 9.8 (2026), pp. 3597–3602. DOI: [10.1021/acsanm.5c05734](https://doi.org/10.1021/acsanm.5c05734).
- [3] Martin Becker, Yan Ravil Wollenweber-Bienerth, S. Hartmann, Angelika Polity, S. Chatterjee, and Peter J. Klar. “Optimization and validation of the thermochromic performance of a trilayer coating of TiO₂/VO₂/TiO₂ for smart windows”. In: *ACS Applied Electronic Materials* 7.16 (2025), pp. 7668–7678. DOI: [10.1021/acsaelm.5c01043](https://doi.org/10.1021/acsaelm.5c01043).
- [4] Guo Li et al. “A thermo-responsive refractive index-matching film for solar modulation of smart windows”. In: *Advanced Energy Materials* (2026), e70975. DOI: [10.1002/aenm.70975](https://doi.org/10.1002/aenm.70975).
- [5] Kai Liu, Sangwook Lee, Shan Yang, Olivier Delaire, and Junqiao Wu. “Recent progresses on physics and applications of vanadium dioxide”. In: *Materials Today* 21.8 (2018), pp. 875–896. DOI: [10.1016/j.mattod.2018.03.029](https://doi.org/10.1016/j.mattod.2018.03.029).
- [6] Amaury Baret, Ambreen Khan, Aline Rougier, Daniel Bellet, and Ngoc Duy Nguyen. “Low-emissivity fine-tuning of efficient VO₂-based thermochromic stacks with silver nanowire networks”. In: *RSC Applied Interfaces* 2.1 (2025), pp. 94–103. DOI: [10.1039/D4LF00234B](https://doi.org/10.1039/D4LF00234B).
- [7] Amaury Baret, Julia Baumgarten, François Balty, Frédéric Rabecki, Jérémy Brisbois, Buyun Zheng, Daniel Bellet, Ngoc Duy Nguyen, et al. “The refractive index of silver nanowire networks: A heuristic approach to the foundations of the optical constants, from experiment to theory”. In: *Discover Nano* 20 (2025), p. 131. DOI: [10.1186/s11671-025-04312-9](https://doi.org/10.1186/s11671-025-04312-9).
- [8] Steven J. Byrnes. *Multilayer optical calculations*. Revised version used by the tmm package. 2016. arXiv: [1603.02720](https://arxiv.org/abs/1603.02720) [[physics.comp-ph](https://arxiv.org/archive/physics)].
- [9] Max Born and Emil Wolf. *Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction of Light*. 7th ed. Cambridge: Cambridge University Press, 2002. ISBN: 9780521642224.
- [10] Shuyi Li, Gunnar A. Niklasson, and Claes G. Granqvist. “Nanothermochromics: Calculations for VO₂ nanoparticles in dielectric hosts show much improved luminous transmittance and solar energy transmittance modulation”. In: *Journal of Applied Physics* 108.6 (2010), p. 063525. DOI: [10.1063/1.3487980](https://doi.org/10.1063/1.3487980).
- [11] John H. Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press, 1992.
- [12] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- [13] Melanie Mitchell. *An Introduction to Genetic Algorithms*. MIT Press, 1998.
- [14] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. 2nd ed. Springer, 2015. DOI: [10.1007/978-3-662-44874-8](https://doi.org/10.1007/978-3-662-44874-8).
- [15] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. “A fast and elitist multiobjective genetic algorithm: NSGA-II”. In: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), pp. 182–197. DOI: [10.1109/4235.996017](https://doi.org/10.1109/4235.996017).